

Имитационное моделирование

Лабораторная работа №2

Исаева Зарина Исмайлбековна

Содержание

1. Цель работы
2. Задание
3. Теоретическое введение
 1. Трёхпараметрическая модель SIR
 2. Модель Лотки–Вольтерры
 3. Численное решение и проверяемые свойства
4. Выполнение лабораторной работы
 1. Подготовка проекта
 2. Общий модуль моделей
 3. Базовая модель SIR
 4. Параметрическое исследование SIR
 5. Базовая модель Лотки–Вольтерры
 6. Параметрическое исследование Лотки–Вольтерры
 7. Литературное программирование и notebooks
 8. Автоматические проверки
5. Выводы
6. Список литературы

Список иллюстраций

1. Структура вычислительного проекта
2. Закреплённые зависимости Julia
3. Базовый сценарий SIR в редакторе
4. Динамика компартментов SIR
5. Эффективное репродуктивное число
6. Фазовая траектория SIR
7. Выполненный notebook базовой SIR
8. Параметрические результаты SIR
9. Выполненный notebook параметрической SIR
10. Параметрический сценарий Лотки–Вольтерры в редакторе
11. Циклы популяций
12. Фазовый портрет хищник–жертва
13. Спектральный анализ
14. Проверка первого интеграла
15. Выполненный notebook базовой модели хищник–жертва
16. Параметрические результаты модели хищник–жертва
17. Выполненный notebook параметрического опыта
18. Генерация производных форматов
19. Результаты автоматических тестов

1. Цель работы

Цель работы — реализовать и исследовать две классические модели, описываемые системами обыкновенных дифференциальных уравнений: эпидемическую модель SIR с явным разделением вероятности передачи и частоты контактов, а также модель взаимодействия популяций Лотки–Вольтерры. Дополнительная цель — оформить вычисления в воспроизводимом литературном проекте Julia.

2. Задание

В соответствии с главой 2 методических указаний [1] необходимо:

1. подготовить рабочий каталог и закрепить зависимости Julia;
2. выполнить предложенные базовые программы `sir_ode.jl` и `lv_ode.jl`;
3. представить оба сценария в литературном стиле;
4. получить из каждого исходника чистый Julia-код, Jupyter Notebook и QMD;
5. выполнить полученные notebooks;
6. добавить отдельные вычисления для наборов параметров обеих моделей;
7. сформировать и выполнить производные варианты параметрических сценариев;
8. интегрировать полные QMD-фрагменты в настоящий отчёт;
9. сохранить таблицы, графики и контрольные показатели, проверить свойства решений автоматическими тестами.

3. Теоретическое введение

3.1 Трёхпараметрическая модель SIR

Классическая модель SIR восходит к работе Кермака и Маккендрика и разделяет закрытую популяцию на восприимчивых S , инфицированных I и удалённых из цепочки передачи R [2,3]. В трёхпараметрической форме вероятность передачи за контакт β отделена от среднего числа контактов c :

$$\begin{aligned}\frac{dS}{dt} &= -\beta c \frac{IS}{N}, \\ \frac{dI}{dt} &= \beta c \frac{IS}{N} - \gamma I, N = S + I + R. (1) \\ \frac{dR}{dt} &= \gamma I,\end{aligned}$$

Базовое репродуктивное число и его эффективное значение имеют вид

$$R_0 = \frac{\beta c}{\gamma}, R_e(t) = R_0 \frac{S(t)}{N}. (2)$$

При $R_0 > 1$ начальное число инфицированных растёт. Пик достигается, когда R_e проходит единицу, а теоретический порог коллективного иммунитета равен $1 - 1/R_0$. Разделение β и c позволяет отдельно интерпретировать биологическую заразность и изменение числа контактов.

3.2 Модель Лотки–Вольтерры

Модель хищник–жертва была сформулирована в работах А. Лотки и В. Вольтерры [4,5]. Она задаётся системой

$$\frac{dx}{dt} = \alpha x - \beta xy, \frac{dy}{dt} = \delta xy - \gamma y, (3)$$

где x — численность жертв, y — численность хищников, α — скорость размножения жертв, β — интенсивность их поедания, δ — эффективность конверсии пищи в прирост хищников, γ — смертность хищников. Положительное равновесие

$$x^* = \frac{\gamma}{\delta}, y^* = \frac{\alpha}{\beta} (4)$$

является центром семейства замкнутых орбит. Вдоль точного решения сохраняется

$$V(x, y) = \delta x - \gamma \ln x + \beta y - \alpha \ln y. (5)$$

Для малых колебаний вблизи равновесия период оценивается как $T \approx 2\pi / \sqrt{\alpha\gamma}$. При удалении начальной точки от равновесия нелинейность увеличивает расхождение этой оценки с наблюдаемым периодом.

3.3 Численное решение и проверяемые свойства

Обе задачи интегрируются адаптивным методом Tsit5 из пакета DifferentialEquations.jl [6]. Для SIR проверяются неотрицательность компонент и закон сохранения $S + I + R = N$. Для модели Лотки–Вольтерры проверяются положительность популяций и малый численный дрейф инварианта

[Уравнение 5](#).

Проектные пути и результаты организованы через DrWatson.jl [7], а чистые скрипты, notebooks и QMD создаются из одного литературного источника с помощью Literate.jl [8].

4. Выполнение лабораторной работы

4.1 Подготовка проекта

Проект разделён на исходный модуль, литературные сценарии, тесты, таблицы, графики и производные форматы.

```
rhktrlwmd@Mac % pwd
/workspace/labs/lab02/project
rhktrlwmd@Mac % find . -maxdepth 2 -type d | sort
.
./data
./markdown
./notebooks
./plots
./scripts
./scripts/lv_ode
./scripts/lv_ode__param
./scripts/sir_ode
./scripts/sir_ode__param
./src
./test
rhktrlwmd@Mac %
```

Рисунок 1: Структура вычислительного проекта

```
rhktrlwmd@Mac % julia --project=. -e 'using Pkg; Pkg.status()'
Status `~/workspace/labs/lab02/project/Project.toml`
 [336ed68f] CSV v0.10.15
 [a93c6f00] DataFrames v1.7.0
 [0c46a032] DifferentialEquations v7.15.0
 [634d3b9d] DrWatson v2.19.1
 [7a1cc6ca] FFTW v1.10.0
 [033835bb] JLD2 v0.5.15
 [98b081ad] Literate v2.20.1
 [91a5bcdd] Plots v1.40.19
rhktrlwmd@Mac %
```

Рисунок 2: Закреплённые зависимости Julia

Основные команды воспроизведения: make setup, make tangle, make run, make notebooks, make docs и make test. Полное описание среды находится в Project.toml.

```
name = "BasicModelsLab"
uuid = "33cdd637-3a8a-4ba9-bf7c-a964e73dcb3e"
authors = ["rhktrlwmd"]
version = "2.0.0"

[deps]
BenchmarkTools = "6e4b80f9-dd63-53aa-95a3-0cdb28fa8baf"
CSV = "336ed68f-0bac-5ca0-87d4-7b16caf5d00b"
DataFrames = "a93c6f00-e57d-5684-b7b6-d8193f3e46c0"
DifferentialEquations = "0c46a032-eb83-5123-abaf-570d42b7fbba"
DrWatson = "634d3b9d-ee7a-5ddf-bec9-22491ea816e1"
FFTW = "7a1cc6ca-52ef-59f5-83cd-3a7055c09341"
IJulia = "7073ff75-c697-5162-941a-fcdaad2a7d2a"
JLD2 = "033835bb-8acc-5ee8-8aae-3f567f8a3819"
LaTeXStrings = "b964fa9f-0449-5b57-a5c2-d3ea65f4040f"
Literate = "98b081ad-f1c9-55d3-8b20-4c87d4299306"
Plots = "91a5bcdd-55d7-5caf-9e0b-520d859cae80"
Quarto = "d7167be5-f61b-4dc9-b75c-ab62374668c5"
Statistics = "10745b16-79ce-11e8-11f9-7d13ad32a3b2"
Tables = "bd369af6-aec1-5ad0-b16a-f7cc5008161c"

[compat]
FFTW = "1.10.0"
Statistics = "1.11.5"
julia = "1.11"
```

Листинг 1 — Полный файл project/Project.toml.

Автоматизация всех этапов записана в Makefile.

```
.PHONY: setup tangle run notebooks docs test all clean-results

SOURCES = scripts/sir_ode.jl scripts/sir_ode__param.jl scripts/lv_ode.jl scripts/lv_ode__param.jl

setup:
    julia --project=. scripts/setup_environment.jl

tangle:
    julia --project=. scripts/tangle.jl $(SOURCES)

run:
    julia --project=. scripts/sir_ode.jl
    julia --project=. scripts/sir_ode__param.jl
    julia --project=. scripts/lv_ode.jl
    julia --project=. scripts/lv_ode__param.jl

notebooks:
    for notebook in notebooks/*/*.ipynb; do \
        jupyter nbconvert --to notebook --execute --inplace \
            --ExecutePreprocessor.kernel_name=julia-lab02-1.11 \
            --ExecutePreprocessor.timeout=900 "$$notebook"; \
    done

docs:
    for document in markdown/*/*.qmd; do \
        quarto render "$$document" --to html --embed-resources; \
    done

test:
    julia --project=. test/runtests.jl

all: tangle run notebooks docs test

clean-results:
    rm -rf data/sir_ode data/sir_ode__param data/lv_ode data/lv_ode__param
    rm -rf plots/sir_ode plots/sir_ode__param plots/lv_ode plots/lv_ode__param
    rm -rf notebooks/sir_ode notebooks/sir_ode__param notebooks/lv_ode notebooks/lv_ode__param
```

```
rm -rf markdown/sir_ode markdown/sir_ode__param markdown/lv_ode markdown/lv_ode__param
```

Листинг 2 — Полный файл project/Makefile.

4.2 Общий модуль моделей

Модуль `BasicModels.jl` отделяет математические правые части и формирование таблиц от конкретных опытов. Это исключает расхождения между базовыми и параметрическими версиями.

```
module BasicModels

using DifferentialEquations
using DataFrames
using Statistics

export sir_rhs!, lotka_volterra_rhs!, solve_sir, solve_lv,
       sir_dataframe, lv_dataframe, first_local_peak, invariant_lv

function sir_rhs!(du, u, p, _t)
    S, I, R = u
    beta, contacts, gamma = p
    N = S + I + R
    infection = beta * contacts * I * S / N
    @inbounds begin
        du[1] = -infection
        du[2] = infection - gamma * I
        du[3] = gamma * I
    end
    return nothing
end

function solve_sir(; u0=[990.0, 10.0, 0.0], p=[0.05, 10.0, 0.25],
                  tspan=(0.0, 40.0), saveat=0.1)
    problem = ODEProblem(sir_rhs!, Float64.(u0), tspan, Float64.(p))
    return solve(problem, Tsit5(); saveat, reltol=1e-9, abstol=1e-11)
end

function sir_dataframe(solution)
    values = reduce(hcat, solution.u)'
    frame = DataFrame(t=solution.t, S=values[:, 1], I=values[:, 2], R=values[:, 3])
    frame.N = frame.S + frame.I + frame.R
    return frame
end

function lotka_volterra_rhs!(du, u, p, _t)
    prey, predator = u
    alpha, beta, delta, gamma = p
    @inbounds begin
        du[1] = alpha * prey - beta * prey * predator
        du[2] = delta * prey * predator - gamma * predator
    end
    return nothing
end

function solve_lv(; u0=[40.0, 9.0], p=[0.1, 0.02, 0.01, 0.3],
                  tspan=(0.0, 200.0), saveat=0.1)
    problem = ODEProblem(lotka_volterra_rhs!, Float64.(u0), tspan, Float64.(p))
    return solve(problem, Tsit5(); saveat, reltol=1e-8, abstol=1e-10)
end

function lv_dataframe(solution, p)
    values = reduce(hcat, solution.u)'
    alpha, beta, delta, gamma = p
    frame = DataFrame(t=solution.t, prey=values[:, 1], predator=values[:, 2])
    frame.dprey_dt = alpha * frame.prey .- beta * frame.prey * frame.predator
end
```



```

frame.dpredator_dt = delta * frame.prey * frame.predator .- gamma * frame.predator
frame.prey_pct_change = 100 * frame.dprey_dt ./ frame.prey
frame.predator_pct_change = 100 * frame.dpredator_dt ./ frame.predator
return frame
end

function first_local_peak(signal, time; start_index=2)
for index in max(start_index, 2):(length(signal) - 1)
if signal[index] > signal[index - 1] && signal[index] > signal[index + 1]
return (time=time[index], value=signal[index], index=index)
end
end
return (time=NaN, value=NaN, index=0)
end

function invariant_lv(pre, predator, p)
alpha, beta, delta, gamma = p
return delta * prey .- gamma * log(prey) .+ beta * predator .- alpha * log(predator)
end

end

```

Листинг 3 — Полный общий модуль project/src/BasicModels.jl.

4.3 Базовая модель SIR

По методичке используются $S_0=990$, $I_0=10$, $R_0^{state}=0$, $\beta=0,05$, $c=10$, $\gamma=0,25$, $t \in [0; 40]$ и шаг сохранения 0,1. Здесь обозначение R_0^{state} относится к начальному числу выздоровевших и не смешивается с репродуктивным числом.

```

Visual Studio Code
lab02 — Visual Studio Code

EXPLORER
LAB02
src
BasicModels.jl
scripts
sir_ode.jl
setup_environment.jl
tangle.jl
test
runtests.jl
Makefile
Project.toml

sir_ode.jl
15
16 # ## Условия эксперимента
17
18 u0 = [990.0, 10.0, 0.0]
19 p = [0.05, 10.0, 0.25] # beta, contacts, gamma
20 tspan = (0.0, 40.0)
21 saveat = 0.1
22 R0_basic = p[1] * p[2] / p[3]
23
24 solution = solve_sir(; u0, p, tspan, saveat)
25 sir = sir_dataframe(solution)
26 sir.Re = R0_basic .* sir.S ./ sir.N
27 peak_index = argmax(sir.I)
28 peak_time = sir.t[peak_index]
29 peak_value = sir.I[peak_index]
30 threshold = 100 * (1 - 1 / R0_basic)
31
32 CSV.write(datadir(script_name, "sir_trajectory.csv"), sir)
33 summary = DataFrame(
34     metric=["R0", "peak_infected", "peak_time", "final_recovered", "population_error"],
35     value=[R0_basic, peak_value, peak_time, sir.R[end], maximum(abs.(sir.N .- sir.N[1]))],
36 )
37 CSV.write(datadir(script_name, "sir_summary.csv"), summary)
38 @save datadir(script_name, "sir_solution.jld2") u0 p tspan solution sir summary
39
40 println("SIR: beta=$(p[1]), c=$(p[2]), gamma=$(p[3]), R0=$(round(R0_basic, digits=3))")
41 println("Peak I=$(round(peak_value, digits=2)) at t=$(round(peak_time, digits=2)) days")
42 println("R(40)=$(round(sir.R[end], digits=2)); population error=$(summary.value[end])")
43
44 # ## Семь представлений динамики
45

```

Рисунок 3: Фрагмент базового сценария SIR в редакторе

Ниже целиком включён QMD, автоматически полученный из литературного исходника scripts/sir_ode.jl.

4.3.1 Базовая трёхпараметрическая модель SIR

Поток заражения разделён на вероятность передачи β и среднее число контактов c . Это позволяет интерпретировать санитарные меры независимо.

```
using DrWatson
@quickactivate "BasicModelsLab"
using BasicModelsLab.BasicModels
using CSV, DataFrames, JLD2, LaTeXStrings, Plots, Statistics

script_name = "sir_ode"
mkpath(datadir(script_name))
mkpath(plotsdir(script_name))
gr()
```

4.3.1.1 Условия эксперимента

```
u0 = [990.0, 10.0, 0.0]
p = [0.05, 10.0, 0.25] # beta, contacts, gamma
tspan = (0.0, 40.0)
saveat = 0.1
R0_basic = p[1] * p[2] / p[3]

solution = solve_sir(; u0, p, tspan, saveat)
sir = sir_dataframe(solution)
sir.Re = R0_basic * sir.S ./ sir.N
peak_index = argmax(sir.I)
peak_time = sir.t[peak_index]
peak_value = sir.I[peak_index]
threshold = 100 * (1 - 1 / R0_basic)

CSV.write(datadir(script_name, "sir_trajectory.csv"), sir)
summary = DataFrame(
    metric=["R0", "peak_infected", "peak_time", "final_recovered", "population_error"],
    value=[R0_basic, peak_value, peak_time, sir.R[end], maximum(abs.(sir.N .- sir.N[1]))],
)
CSV.write(datadir(script_name, "sir_summary.csv"), summary)
@save datadir(script_name, "sir_solution.jld2") u0 p tspan solution sir summary

println("SIR: beta=$(p[1]), c=$(p[2]), gamma=$(p[3]), R0=$(round(R0_basic, digits=3))")
println("Peak I=$(round(peak_value, digits=2)) at t=$(round(peak_time, digits=2)) days")
println("R(40)=$(round(sir.R[end], digits=2)); population error=$(summary.value[end])")
```

4.3.1.2 Семь представлений динамики

```
main_plot = plot(sir.t, [sir.S sir.I sir.R];
    label=[L"S(t)" L"I(t)" L"R(t)"], xlabel="Время, дни",
    ylabel="Число людей", title="SIR: динамика эпидемии", lw=2,
    color=[:royalblue :firebrick :seagreen], size=(900, 520), legend=:right)

infected_plot = plot(sir.t, sir.I; label=L"I(t)", xlabel="Время, дни",
    ylabel="Инфицированные", title="Число инфицированных", color=:firebrick,
    lw=2.5, fill=(0, 0.18, :firebrick), size=(900, 440))
vline!(infected_plot, [peak_time]; label="пик $(round(peak_value, digits=1))",
    color=:black, ls=:dash)

log_plot = plot(sir.t, sir.I; label=L"I(t)", xlabel="Время, дни",
    ylabel="Инфицированные", title="Начальный рост в логарифмической шкале",
    yscale=:log10, color=:firebrick, lw=2, size=(900, 440))

percentage_plot = plot(sir.t, 100 * Matrix(select(sir, [:S, :I, :R])) ./ sir.N;
    label=[L"S/N" L"I/N" L"R/N"], xlabel="Время, дни", ylabel="Доля, %",
    title="Доли компартментов", lw=2, color=[:royalblue :firebrick :seagreen],
    size=(900, 520), legend=:right)
hline!(percentage_plot, [threshold]; label="порог иммунитета $(round(threshold, digits=1))%",
    color=:purple, ls=:dash)

phase_plot = plot(sir.S, sir.I; label="траектория", xlabel=L"S(t)", ylabel=L"I(t)",
```

```

title="Фазовый портрет SIR", color=:navy, lw=2.2, size=(820, 540))
scatter!(phase_plot, [sir.S[1], sir.S[end]], [sir.I[1], sir.I[end]]);
label=["начало" "t=40"], color=[:green :black], ms=5)

effective_plot = plot(sir.t, sir.Re; label=L"R_e(t)", xlabel="Время, дни",
    ylabel=L"R_e", title="Эффективное репродуктивное число", color=:seagreen,
    lw=2.3, size=(900, 440))
hline!(effective_plot, [1.0]; label=L"R_e=1", color=:firebrick, ls=:dash)

panel = plot(main_plot, infected_plot, log_plot, percentage_plot,
    phase_plot, effective_plot; layout=(3, 2), size=(1300, 1100))

for (plot_object, filename) in [
    (main_plot, "sir_main.png"), (infected_plot, "sir_infected.png"),
    (log_plot, "sir_log_scale.png"), (percentage_plot, "sir_percentages.png"),
    (phase_plot, "sir_phase_portrait.png"), (effective_plot, "sir_effective_R.png"),
    (panel, "sir_panel.png")]
    savefig(plot_object, plotsdir(script_name, filename))
end

display(panel)

```

Листинг 4 — Полный литературный сценарий базовой модели SIR.

Для выбранных параметров $R_0 = 2$, теоретический порог иммунитета равен 50%. Численный пик составляет 158,45 инфицированного и наблюдается на 17,5-й день. К моменту $t = 40$ в компоненте R находится 775,69 человека.

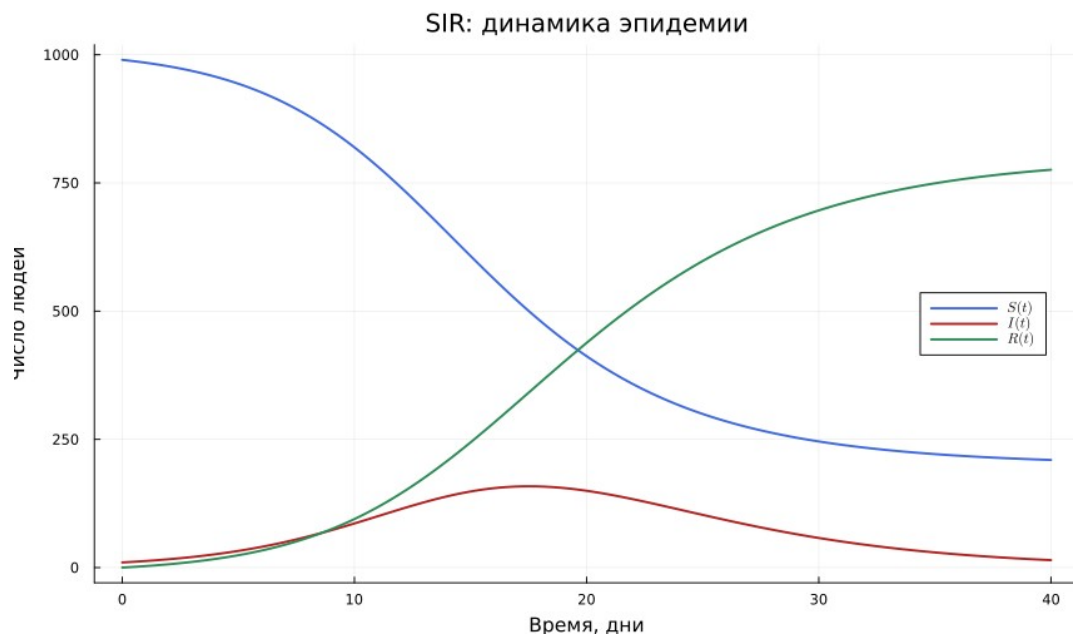


Рисунок 4: Динамика трёх компартментов SIR

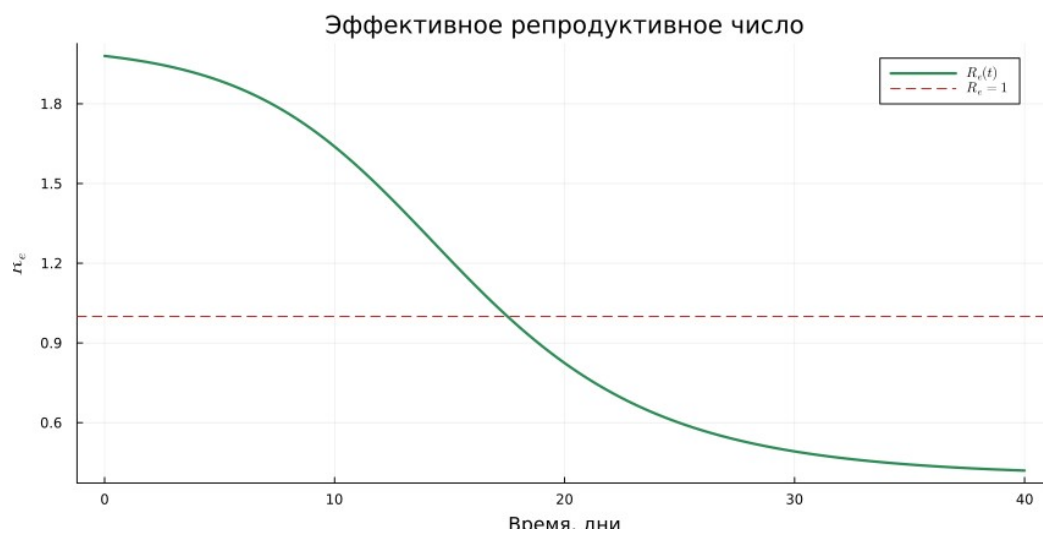


Рисунок 5: Эффективное репродуктивное число

Фазовая траектория показывает однозначное уменьшение S и рост I до пересечения порога $S/N = 1/R_0$, после чего I снижается.

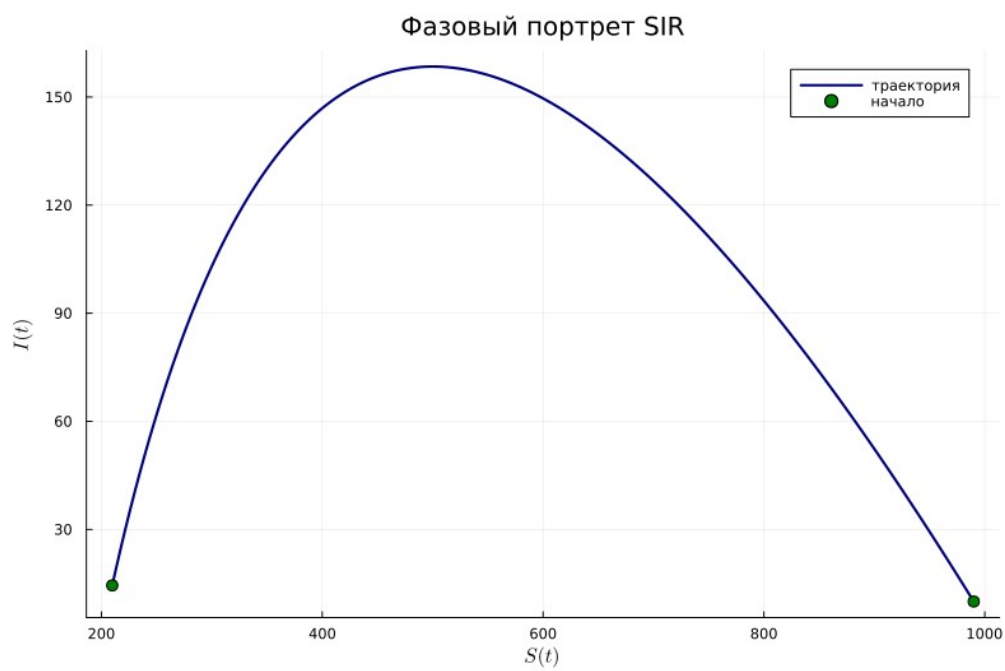


Рисунок 6: Фазовая траектория SIR

Базовая трёхпараметрическая модель SIR

Поток заражения разделён на вероятность передачи β и среднее число контактов c . Это позволяет интерпретировать санитарные меры независимо.

```
In [1]: using DrWatson
@quickactivate "BasicModelsLab"
using BasicModelsLab.BasicModels
using CSV, DataFrames, JLD2, LaTeXStrings, Plots, Statistics

script_name = "sir_ode"
mkpath(datadir(script_name))
mkpath(plotsdir(script_name))
gr()
```

Out [1]: Plots.GRBackend()

Условия эксперимента

```
In [2]: u0 = [990.0, 10.0, 0.0]
p = [0.05, 10.0, 0.25] # beta, contacts, gamma
tspan = (0.0, 40.0)
saveat = 0.1
R0_basic = p[1] * p[2] / p[3]

solution = solve_sir(; u0, p, tspan, saveat)
sir = sir_dataframe(solution)
sir.Re = R0_basic .* sir.S ./ sir.N
peak_index = argmax(sir.I)
peak_time = sir.t[peak_index]
peak_value = sir.I[peak_index]
threshold = 100 * (1 - 1 / R0_basic)

CSV.write(datadir(script_name), "sir_trajectory.csv", sir)

SIR: beta=0.05, c=10.0, gamma=0.25, R0=2.0
Peak I=158.45 at t=17.5 days
R(40)=775.69; population error=6.821210263296962e-13
```

Семь представлений динамики

```
In [3]: main_plot = plot(sir.t, [sir.S sir.I sir.R];
    label=["S(t)" "I(t)" "R(t)"], xlabel="Время, дни",
    ylabel="Число людей", title="SIR: динамика эпидемии", lw=2,
    color=:royalblue :firebrick :seagreen, size=(900, 520), legend=:right)

infected_plot = plot(sir.t, sir.I; label="I(t)", xlabel="Время, дни",
    ylabel="Инфицированные", title="Число инфицированных", color=:firebrick,
    lw=2.5, fill=(0, 0.18, :firebrick), size=(900, 440))
vline!(infected_plot, [peak_time]; label="пик $(round(peak_value, digits=1))",
```

Рисунок 7: Выполненный notebook базовой SIR

4.4 Параметрическое исследование SIR

Отдельный полный факторный план содержит 36 сочетаний:

- $\beta \in \{0,025; 0,04; 0,055; 0,07\}$;
- $c \in \{5; 10; 15\}$ контактов в день;
- $\gamma \in \{0,15; 0,25; 0,35\}$.

Для каждого опыта сохранены R_0 , пик и время пика, $R(60)$, $S(60)$ и ошибка баланса населения.

4.4.1 Параметрическое исследование модели SIR

Полный факторный план разделяет влияние вероятности передачи, частоты контактов и скорости выздоровления на пик эпидемии.

```
using DrWatson
@quickactivate "BasicModelsLab"
using BasicModelsLab.BasicModels
using CSV, DataFrames, JLD2, Plots, Statistics

script_name = "sir_ode_param"
mkpath(datadir(script_name))
mkpath(plotsdir(script_name))
gr()

betas = [0.025, 0.04, 0.055, 0.07]
contacts = [5.0, 10.0, 15.0]
gammas = [0.15, 0.25, 0.35]
```

```

u0 = [990.0, 10.0, 0.0]
tspan = (0.0, 60.0)
rows = namedtuple([],)

for beta in betas, contacts_per_day in contacts, gamma in gammas
    p = [beta, contacts_per_day, gamma]
    frame = sir_dataframe(solve_sir; u0, p, tspan, saveat=0.2)
    peak_index = argmax(frame.I)
    push!(rows, (
        beta=beta, contacts=contacts_per_day, gamma=gamma,
        R0=beta * contacts_per_day / gamma,
        peak_infected=frame.I[peak_index], peak_time=frame.t[peak_index],
        final_recovered=frame.R[end], susceptible_end=frame.S[end],
        population_error=maximum(abs.(frame.N .- sum(u0))),
    ))
end

scan = DataFrame(rows)
CSV.write(datadir(script_name, "sir_parameter_grid.csv"), scan)
@save datadir(script_name, "sir_parameter_grid.jld2") betas contacts gammas scan

println("SIR parameter grid: $(nrow(scan)) combinations")
println("R0 range: $(round(minimum(scan.R0), digits=3))–$(round(maximum(scan.R0), digits=3))")
println("Peak I range: $(round(minimum(scan.peak_infected), digits=2))–$(round(maximum(scan.peak_infected), digits=2))")

reference_gamma = 0.25
slice = filter(:gamma == reference_gamma, scan)
peak_heat = reshape(slice.peak_infected, length(betas), length(contacts))
final_heat = reshape(slice.final_recovered, length(betas), length(contacts))

peak_plot = heatmap(contacts, betas, peak_heat;
    xlabel="Контакты c", ylabel="Вероятность beta", colorbar_title="I max",
    title="Пик эпидемии при gamma=$(reference_gamma)", c=:thermal, size=(820, 520))
final_plot = heatmap(contacts, betas, final_heat;
    xlabel="Контакты c", ylabel="Вероятность beta", colorbar_title="R(60)",
    title="Итоговое число переболевших", c=:viridis, size=(820, 520))

r0_plot = scatter(scan.R0, scan.peak_infected; group=string.(scan.gamma),
    xlabel="R0", ylabel="Пиковое I", title="Пик и репродуктивное число",
    label=["gamma=.15" "gamma=.25" "gamma=.35"], ms=5, alpha=.8, size=(820, 520))

duration_plot = scatter(1 ./ scan.gamma, scan.peak_time; group=string.(scan.contacts),
    xlabel="Средняя длительность болезни, дни", ylabel="Время пика, дни",
    title="Момент пика", ms=5, alpha=.8, size=(820, 520),
    label=["c=5" "c=10" "c=15"])

panel = plot(peak_plot, final_plot, r0_plot, duration_plot; layout=(2, 2), size=(1300, 920))
savefig(peak_plot, plotsdir(script_name, "sir_peak_heatmap.png"))
savefig(final_plot, plotsdir(script_name, "sir_final_heatmap.png"))
savefig(r0_plot, plotsdir(script_name, "sir_peak_vs_R0.png"))
savefig(duration_plot, plotsdir(script_name, "sir_peak_timing.png"))
savefig(panel, plotsdir(script_name, "sir_parameter_panel.png"))

display(panel)

```

Листинг 5 — Полный литературный сценарий параметрической SIR.

Таблица 1: Диапазоны результатов 36 опытов SIR

Показатель	Минимум	Максимум
R_0	0,357	7,000
пик I	10,00	580,35
время пика	0,0	50,6
$R(60)$	15,45	998,79

При $R_0 < 1$ максимум равен начальному $I_0 = 10$ и возникает при $t = 0$. Наиболее интенсивная вспышка получена для $\beta = 0,07$, $c = 15$, $\gamma = 0,15$: $R_0 = 7$, пик 580,35 на 7,2-й день, $R(60) = 998,79$.

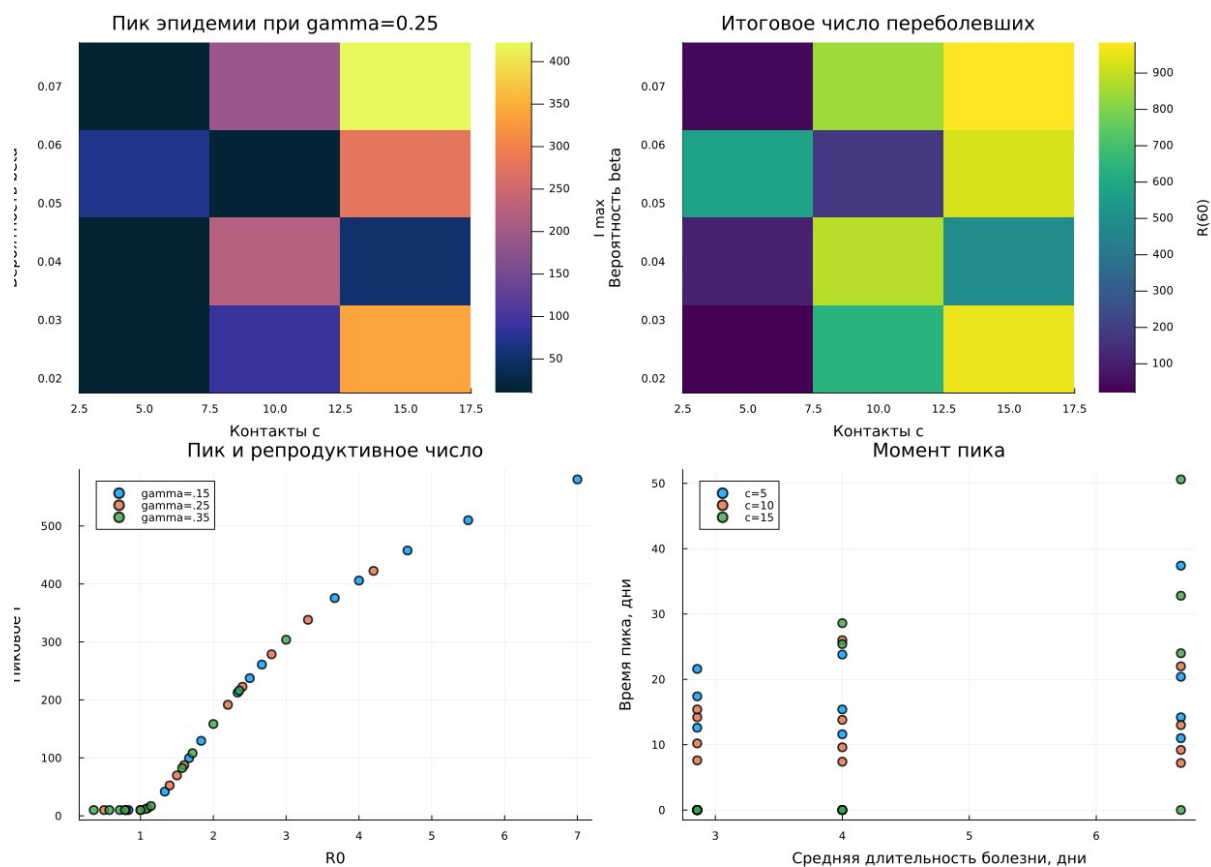


Рисунок 8: Сводка параметрических опытов SIR

Параметрическое исследование модели SIR

Полный факторный план разделяет влияние вероятности передачи, частоты контактов и скорости выздоровления на пик эпидемии.

```
In [1]: using DrWatson
@quickactivate "BasicModelsLab"
using BasicModelsLab.BasicModels
using CSV, DataFrames, JLD2, Plots, Statistics

script_name = "sir_ode_param"
mkpath(datadir(script_name))
mkpath(plotsdir(script_name))
gr()

betas = [0.025, 0.04, 0.055, 0.07]
contacts = [5.0, 10.0, 15.0]
gammas = [0.15, 0.25, 0.35]
u0 = [990.0, 10.0, 0.0]
tspan = (0.0, 60.0)

SIR parameter grid: 36 combinations
R0 range: 0.357-7.0
Peak I range: 10.0-580.35
Пик эпидемии при gamma=0.25
```

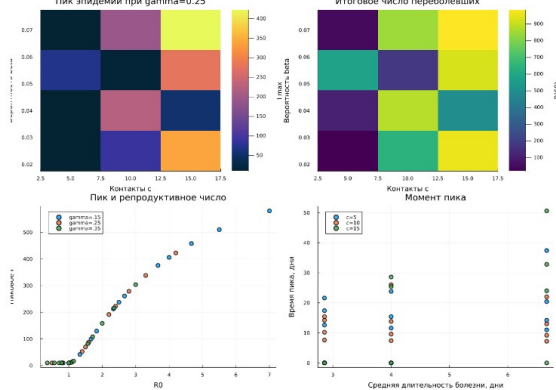


Рисунок 9: Выполненный notebook параметрической SIR

4.5 Базовая модель Лотки–Вольтерры

Базовые параметры строго соответствуют примеру: $\alpha=0,1$, $\beta=0,02$, $\delta=0,01$, $\gamma=0,3$, $x_0=40$, $y_0=9$, $t \in [0; 200]$, шаг сохранения 0,1. Равновесие равно (30; 5).

4.5.1 Базовая модель Лотки–Вольтерры

Классическая система описывает циклическое взаимодействие популяций жертв и хищников. Численно проверяются равновесие, фазовый сдвиг и первый интеграл.

```
using DrWatson
@quickactivate "BasicModelsLab"
using BasicModelsLab.BasicModels
using CSV, DataFrames, FFTW, JLD2, LaTeXStrings, Plots, Statistics

script_name = "lv_ode"
mkpath(datadir(script_name))
mkpath(plotsdir(script_name))
gr()

p = [0.1, 0.02, 0.01, 0.3] # alpha, beta, delta, gamma
u0 = [40.0, 9.0]
tspan = (0.0, 200.0)
saveat = 0.1

solution = solve_lv(; u0, p, tspan, saveat)
lv = lv_dataframe(solution, p)
lv.invariant = invariant_lv(lv.prey, lv.predator, p)
x_star = p[4] / p[3]
y_star = p[1] / p[2]
```



```

theoretical_period = 2pi / sqrt(p[1] * p[4])
prey_peak = first_local_peak(lv.prey, lv.t)
predator_peak = first_local_peak(lv.predator, lv.t; start_index=prey_peak.index + 1)
phase_lag = predator_peak.time - prey_peak.time
invariant_drift = maximum(abs.(lv.invariant .- lv.invariant[1]))

CSV.write(datadir(script_name, "lv_trajectory.csv"), lv)
summary = DataFrame(
    metric=["x_star", "y_star", "period_theory", "prey_peak_time",
            "predator_peak_time", "phase_lag", "invariant_drift"],
    value=[x_star, y_star, theoretical_period, prey_peak.time,
            predator_peak.time, phase_lag, invariant_drift],
)
CSV.write(datadir(script_name, "lv_summary.csv"), summary)
@save datadir(script_name, "lv_solution.jld2") p u0 solution lv summary

println("Lotka-Volterra: alpha=$(p[1]), beta=$(p[2]), delta=$(p[3]), gamma=$(p[4])")
println("Equilibrium: x*=$(x_star), y*=$(y_star)")
println("First peaks: prey t=$(prey_peak.time), predator t=$(predator_peak.time), lag=$(phase_lag)")
println("Invariant drift=$(invariant_drift)")

dynamics = plot(lv.t, [lv.prey lv.predator];
    label=["Жертвы x(t)" "Хищники y(t)", xlabel="Время",
    ylabel="Популяция", title="Динамика популяций", lw=2.2,
    color=:seagreen :firebrick], size=(900, 520))
hline!(dynamics, [x_star, y_star]; label=["x*" "y*"],
    color=:darkgreen :darkred, ls=:dash, alpha=.65)

phase = plot(lv.prey, lv.predator; label="траектория", xlabel="Жертвы x",
    ylabel="Хищники y", title="Фазовый портрет", color=:navy, lw=1.8,
    size=(820, 560))
scatter!(phase, [x_star], [y_star]; label="равновесие", color=:black, ms=6)
vline!(phase, [x_star]; label="dx/dt=0", color=:seagreen, ls=:dash)
hline!(phase, [y_star]; label="dy/dt=0", color=:firebrick, ls=:dash)

derivatives = plot(lv.t, [lv.dprey_dt lv.dpredator_dt];
    label=["L dx/dt" "L dy/dt"], xlabel="Время", ylabel="Скорость",
    title="Скорости изменения", color=:seagreen :firebrick, lw=1.6,
    size=(900, 440))
hline!(derivatives, [0.0]; label="", color=:black, alpha=.4)

relative = plot(lv.t, [lv.prey_pct_change lv.predator_pct_change];
    label=["L 100x'/x" "L 100y'/y"], xlabel="Время", ylabel="Изменение, %",
    title="Относительные темпы роста", color=:seagreen :firebrick,
    lw=1.6, size=(900, 440))

function positive_spectrum(values, step)
    centered = values .- mean(values)
    amplitude = abs.(rfft(centered))
    frequency = rfftfreq(length(centered), inv(step))
    return frequency[2:end], amplitude[2:end]
end

frequency, prey_spectrum = positive_spectrum(lv.prey, saveat)
_, predator_spectrum = positive_spectrum(lv.predator, saveat)
dominant_index = argmax(prey_spectrum)
observed_period = 1 / frequency[dominant_index]
spectrum = plot(frequency, [prey_spectrum predator_spectrum];
    label=["жертвы" "хищники"], xlabel="Частота", ylabel="Амплитуда",
    title="Спектральный анализ", xscale=:log10, yscale=:log10,
    color=:seagreen :firebrick, lw=1.8, size=(900, 440))

invariant_plot = plot(lv.t, lv.invariant .- lv.invariant[1];
    label="V(t)-V(0)", xlabel="Время", ylabel="Отклонение",
    title="Численное сохранение первого интеграла", color=:purple,
    lw=1.8, size=(900, 440))

panel = plot(dynamics, phase, derivatives, relative, spectrum, invariant_plot;
    layout=(3, 2), size=(1300, 1100))
for (plot_object, filename) in [
    (dynamics, "lv_dynamics.png"), (phase, "lv_phase_portrait.png"),

```

```

(derivatives, "lv_derivatives.png"), (relative, "lv_relative_changes.png"),
(spectrum, "lv_spectrum.png"), (invariant_plot, "lv_invariant.png"),
(panel, "lv_panel.png")]
savefig(plot_object, plotsdir(script_name, filename))
end

println("Periods: theory=$(round(theoretical_period, digits=3)), spectrum=$(round(observed_period, digits=3))")
display(panel)

```

Листинг 6 — Полный литературный сценарий базовой модели Лотки–Вольтерры.

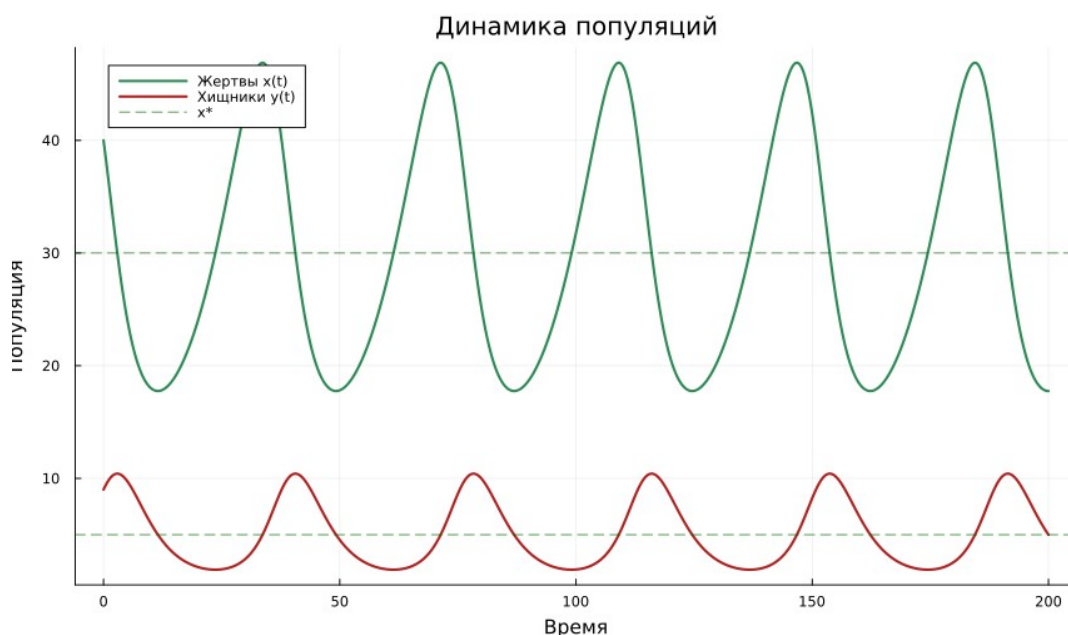


Рисунок 10: Циклы численности жертв и хищников

Первый показанный пик жертв приходится на $t=33,7$, следующий пик хищников — на $t=40,6$. Положительный лаг 6,9 подтверждает ожидаемую последовательность: сначала растёт кормовая база, затем увеличивается популяция хищников.

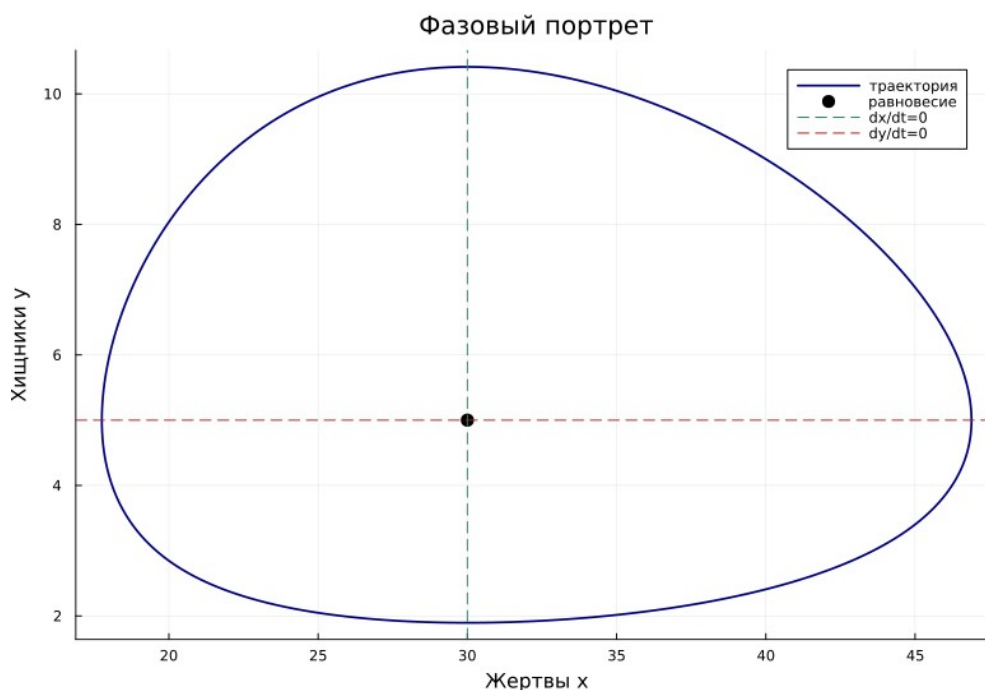


Рисунок 11: Замкнутая орбита вокруг равновесия

Спектральная оценка периода равна 40,02, тогда как локальная линейная оценка даёт 36,276. Разница объясняется конечной амплитудой орбиты.

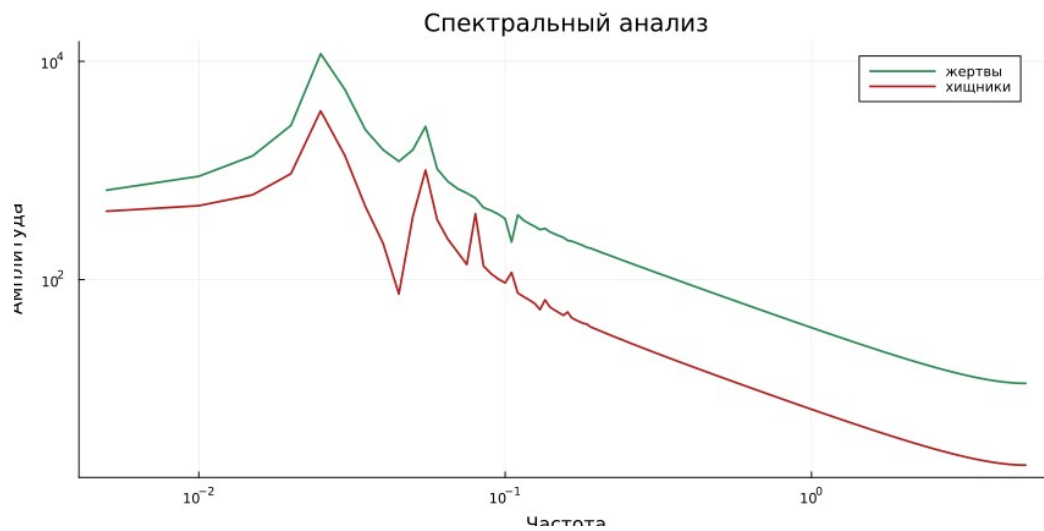


Рисунок 12: Спектральный анализ популяций

Максимальный численный дрейф $V(t) - V(0)$ равен $1,27 \cdot 10^{-9}$, поэтому замкнутая орбита не является следствием заметного численного накопления ошибки.

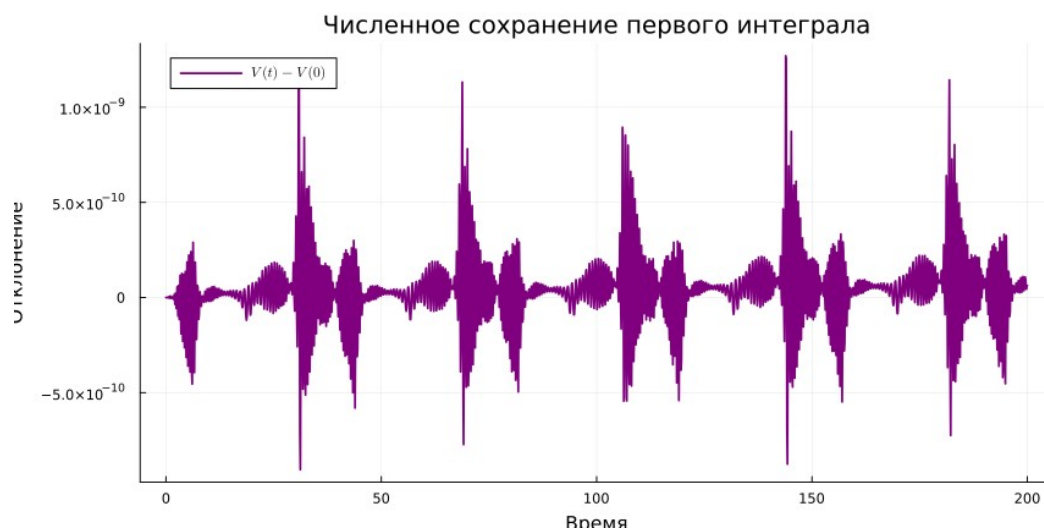


Рисунок 13: Проверка сохранения первого интеграла

Базовая модель Лотки–Вольтерры

Классическая система описывает циклическое взаимодействие популяций жертв и хищников. Численно проверяются равновесие, фазовый сдвиг и первый интеграл.

```
In [1]: using DrWatson
@quickactivate "BasicModelsLab"
using BasicModelsLab.BasicModels
using CSV, DataFrames, FFTW, JLD2, LaTeXStrings, Plots, Statistics

script_name = "lv_ode"
mkpath(datadir(script_name))
mkpath(plotsdir(script_name))
gr()

p = [0.1, 0.02, 0.01, 0.3] # alpha, beta, delta, gamma
u0 = [40.0, 9.0]
tspan = (0.0, 200.0)
saveat = 0.1
```

Lotka-Volterra: alpha=0.1, beta=0.02, delta=0.01, gamma=0.3
Equilibrium: x=30.0, y=5.0
First peaks: prey t=33.7, predator t=40.6, lag=6.899999999999999
Invariant drift=1.2698360141172316e-9
Periods: theory=36.276, spectrum=40.02
Динамика популяций

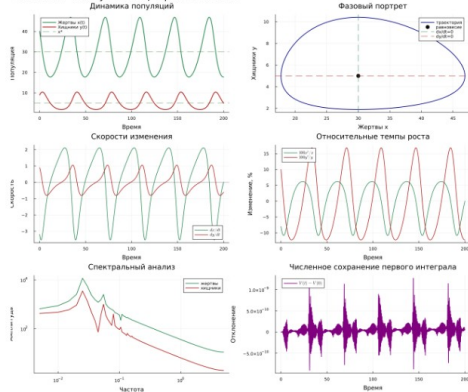


Рисунок 14: Выполненный notebook базовой модели Лотки–Вольтерры

4.6 Параметрическое исследование Лотки–Вольтерры

Согласно предложенному в методичке набору исследуются $\alpha \in \{0,05; 0,1; 0,2; 0,3\}$ и $\gamma \in \{0,1; 0,3; 0,5; 0,7\}$. Параметры взаимодействия $\beta=0,02$ и $\delta=0,01$ фиксированы. Полный план содержит 16 опытов, горизонт 600 обеспечивает не менее двух циклов во всех режимах.

```
Visual Studio Code
lab02 — Visual Studio Code

lv_ode_param.jl
12 mkpath(datadir(script_name))
13 mkpath(plotsdir(script_name))
14 gr()
15
16 alphas = [0.05, 0.1, 0.2, 0.3]
17 gammas = [0.1, 0.3, 0.5, 0.7]
18 beta = 0.02
19 delta = 0.01
20 u0 = [40.0, 9.0]
21 rows = NamedTuple[]
22
23 function estimate_period(signal, time)
24     peak_times = Float64[]
25     for index in 2:(length(signal) - 1)
26         signal[index] > signal[index - 1] && signal[index] > signal[index + 1] &&
27             push!(peak_times, time[index])
28     end
29     return length(peak_times) >= 2 ? mean(diff(peak_times)) : NaN
30 end
31
32 for alpha in alphas, gamma in gammas
33     p = [alpha, beta, delta, gamma]
34     frame = lv_dataframe(solve_lv(; u0, p, tspan=(0.0, 600.0), saveat=0.2), p)
35     conserved = invariant_lv(frame.prey, frame.predator, p)
36     push!(rows, (
37         alpha=alpha, gamma=gamma, x_star=gamma / delta, y_star=alpha / beta,
38         period_theory=2pi / sqrt(alpha * gamma),
39         period_observed=estimate_period(frame.prey, frame.t),
40         prey_min=minimum(frame.prey), prey_max=maximum(frame.prey),
41         predator_min=minimum(frame.predator), predator_max=maximum(frame.predator),
42         invariant_drift=maximum(abs.(conserved .- conserved[1])),
```

Рисунок 15: Фрагмент параметрического сценария в редакторе

4.6.1 Параметрическое исследование модели Лотки–Вольтерры

В полном плане изменяются рождаемость жертв α и смертность хищников γ . Для каждого опыта измеряются период, амплитуды и сохранение интеграла.

```
using DrWatson
@quickactivate "BasicModelsLab"
using BasicModelsLab.BasicModels
using CSV, DataFrames, JLD2, Plots, Statistics

script_name = "lv_ode_param"
mkpath(datadir(script_name))
mkpath(plotsdir(script_name))
gr()

alphas = [0.05, 0.1, 0.2, 0.3]
gammas = [0.1, 0.3, 0.5, 0.7]
beta = 0.02
delta = 0.01
u0 = [40.0, 9.0]
rows = NamedTuple[]

function estimate_period(signal, time)
    peak_times = Float64[]
    for index in 2:(length(signal) - 1)
        signal[index] > signal[index - 1] && signal[index] > signal[index + 1] &&
            push!(peak_times, time[index])
    end
    return length(peak_times) >= 2 ? mean(diff(peak_times)) : NaN
end

for alpha in alphas, gamma in gammas
    p = [alpha, beta, delta, gamma]
    frame = lv_dataframe(solve_lv(; u0, p, tspan=(0.0, 600.0), saveat=0.2), p)
    conserved = invariant_lv(frame.prey, frame.predator, p)
    push!(rows, (
        alpha=alpha, gamma=gamma, x_star=gamma / delta, y_star=alpha / beta,
        period_theory=2pi / sqrt(alpha * gamma),
        period_observed=estimate_period(frame.prey, frame.t),
        prey_min=minimum(frame.prey), prey_max=maximum(frame.prey),
        predator_min=minimum(frame.predator), predator_max=maximum(frame.predator),
        invariant_drift=maximum(abs.(conserved .- conserved[1])),
    ))
end

scan = DataFrame(rows)
CSV.write(datadir(script_name, "lv_parameter_grid.csv"), scan)
@save datadir(script_name, "lv_parameter_grid.jld2") alphas gammas beta delta scan

println("Lotka–Volterra parameter grid: $(nrow(scan)) combinations")
println("Observed periods: $(round(minimum(scan.period_observed), digits=2))–$(round(maximum(scan.period_observed), digits=2))")
println("Maximum invariant drift: $(maximum(scan.invariant_drift))")

period_matrix = reshape(scan.period_observed, length(gammas), length(alphas))
prey_amplitude = reshape(scan.prey_max .- scan.prey_min, length(gammas), length(alphas))
predator_amplitude = reshape(scan.predator_max .- scan.predator_min, length(gammas), length(alphas))

period_heat = heatmap(gammas, alphas, period_matrix;
    xlabel="Смертность хищников gamma", ylabel="Рождаемость жертв alpha",
    colorbar_title="Период", title="Наблюдаемый период циклов", c=:viridis,
    size=(820, 520))
```

```

prey_heat = heatmap(gammas, alphas, prey_amplitude;
    xlabel="gamma", ylabel="alpha", colorbar_title="max(x)-min(x)",
    title="Амплитуда популяции жертв", c=:greens, size=(820, 520))
predator_heat = heatmap(gammas, alphas, predator_amplitude;
    xlabel="gamma", ylabel="alpha", colorbar_title="max(y)-min(y)",
    title="Амплитуда популяции хищников", c=:reds, size=(820, 520))
agreement = scatter(scan.period_theory, scan.period_observed;
    xlabel="Теоретический период", ylabel="Наблюдаемый период",
    title="Сопоставление периодов", label="опыты", color=:purple,
    ms=6, alpha=.8, size=(820, 520))
limits = extrema(vcat(scan.period_theory, scan.period_observed))
plot!(agreement, collect(limits), collect(limits); label="равенство", color=:black, ls=:dash)

panel = plot(period_heat, prey_heat, predator_heat, agreement;
    layout=(2, 2), size=(1300, 920))
savefig(period_heat, plotsdir(script_name, "lv_period_heatmap.png"))
savefig(prey_heat, plotsdir(script_name, "lv_prey_amplitude.png"))
savefig(predator_heat, plotsdir(script_name, "lv_predator_amplitude.png"))
savefig(agreement, plotsdir(script_name, "lv_period_agreement.png"))
savefig(panel, plotsdir(script_name, "lv_parameter_panel.png"))

display(panel)

```

Листинг 7 — Полный литературный сценарий параметрической модели Лотки–Вольтерры.

Таблица 2: Диапазоны результатов 16 опытов Лотки–Вольтерры

Показатель	Минимум	Максимум
наблюдаемый период	14,40	142,93
максимум жертв	40,14	128,03
максимум хищников	9,79	39,10
дрейф инварианта	$1,40 \cdot 10^{-9}$	$9,87 \cdot 10^{-9}$

Самый короткий период наблюдается при $\alpha=0,3$, $\gamma=0,7$; самый длинный — при $\alpha=0,05$, $\gamma=0,1$. Направление изменения согласуется с зависимостью $2\pi/\sqrt{\alpha\gamma}$, а различия по величине характеризуют нелинейный режим конечной амплитуды.

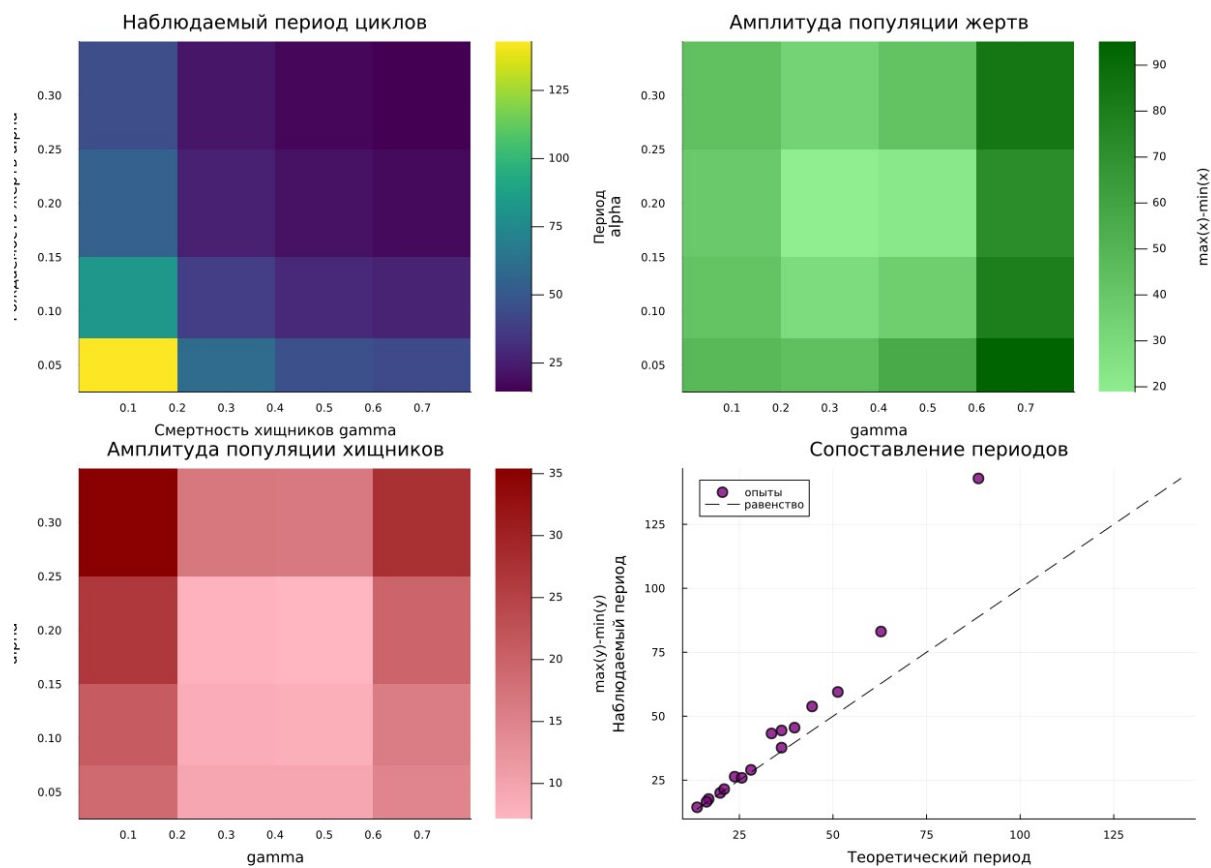


Рисунок 16: Параметрические результаты модели хищник–жертва

Параметрическое исследование модели Лотки–Вольтерры

В полном плане изменяются рождаемость жертв α и смертность хищников γ . Для каждого опыта измеряются период, амплитуды и сохранение интеграла.

```
In [1]: using DrWatson
@quickactivate "BasicModelsLab"
using BasicModelsLab.BasicModels
using CSV, DataFrames, JLD2, Plots, Statistics

script_name = "lv_ode_param"
mkpath(datadir(script_name))
mkpath(plotsdir(script_name))
gr()

alphas = [0.05, 0.1, 0.2, 0.3]
gammas = [0.1, 0.3, 0.5, 0.7]
beta = 0.02
delta = 0.01
u0 = [40.0, 9.0]
```

Lotka–Volterra parameter grid: 16 combinations
Observed periods: 14.4–142.93
Maximum invariant drift: 9.866979233663642e-9

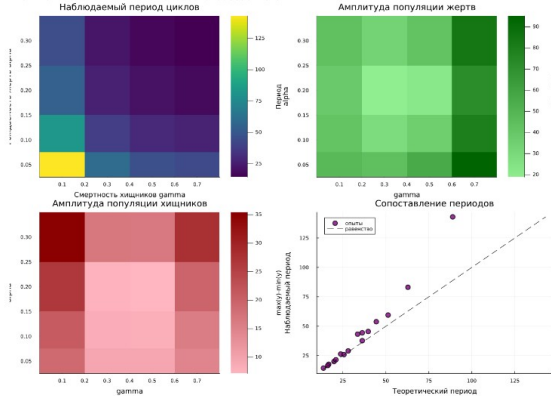


Рисунок 17: Выполненный параметрический notebook

4.7 Литературное программирование и производные форматы

Генератор создаёт для каждого из четырёх литературных исходников чистый скрипт, notebook и QMD. Логика преобразования приведена полностью.

```
#!/usr/bin/env julia

using DrWatson
@quickactivate "BasicModelsLab"
using Literate

function without_execution_markers(content)
    replace(content, r"(?m)^\s?>" => "")
end

function generate_formats(source::AbstractString)
    isfile(source) || error("Source file not found: $(source)")
    name = splitext(basename(source))[1]
    clean_dir = scriptsdir(name)
    quarto_dir = projectdir("markdown", name)
    notebook_dir = projectdir("notebooks", name)
    mkpath([clean_dir, quarto_dir, notebook_dir])
    Literate.script(source, clean_dir; name, credit=false)
    Literate.notebook(source, notebook_dir; name, execute=false, credit=false)
    Literate.markdown(source, quarto_dir; name,
        flavor=Literate.QuartoFlavor(), credit=false,
        preprocess=without_execution_markers)
    println("Generated Julia, notebook, and Quarto formats for $(name)")
end

isempty(ARGS) && error("Usage: julia --project=. scripts/tangle.jl <source.jl> ...")
foreach(generate_formats, ARGS)
```

Листинг 8 — Полный генератор `project/scripts/tangle.jl`.

Ядро Jupyter устанавливается в то же изолированное окружение проекта.

```
#!/usr/bin/env julia

using Pkg
Pkg.activate(joinpath(@__DIR__, ".."))
Pkg.instantiate()
Pkg.precompile()

using IJulia
ENV["JUPYTER_DATA_DIR"] = joinpath(first(DEPOT_PATH), "jupyter")
IJulia.installkernel("Julia lab02", "--project=$(abspath(joinpath(@__DIR__, "..")))")
println("Julia environment and kernel julia-lab02-1.11 are ready")
```

Листинг 9 — Полная настройка окружения `setup_environment.jl`.


```

rhktrlwmd@Mac % make tangle
Generated Julia, notebook, and Quarto formats for sir_ode
Generated Julia, notebook, and Quarto formats for sir_ode__param
Generated Julia, notebook, and Quarto formats for lv_ode
Generated Julia, notebook, and Quarto formats for lv_ode__param
rhktrlwmd@Mac % make docs
Output created: lv_ode.html
Output created: lv_ode__param.html
Output created: sir_ode.html
Output created: sir_ode__param.html
rhktrlwmd@Mac %

```

Рисунок 18: Фактическое формирование производных форматов

Все четыре IPYNB выполнены ядром `julia-lab02-1.11`. В них отсутствуют ошибки, каждый содержит сохранённый текстовый результат и итоговый график. HTML для каждого QMD собран с внедрёнными ресурсами.

4.8 Автоматические проверки

Тесты SIR контролируют 401 временную точку, неотрицательность компонент, сохранение населения, существование эпидемического пика и направление потоков. Тесты Лотки–Вольтерры проверяют 2001 точку, положительность популяций, наличие локальных пиков и сохранение первого интеграла.

```

using Test
using BasicModelsLab.BasicModels

@testset "SIR model" begin
    p = [0.05, 10.0, 0.25]
    df = sir_dataframe(solve_sir(; p))
    @test length(df.t) == 401
    @test maximum(abs.(df.N .- 1000.0)) < 1e-6
    @test all(df.S .>= 0) && all(df.I .>= 0) && all(df.R .>= 0)
    @test maximum(df.I) > 150
    @test df.S[end] < df.S[1] && df.R[end] > df.R[1]
end

@testset "Lotka–Volterra model" begin
    p = [0.1, 0.02, 0.01, 0.3]
    df = lv_dataframe(solve_lv(; p), p)
    conserved = invariant_lv(df.prey, df.predator, p)
    @test length(df.t) == 2001
    @test all(df.prey .> 0) && all(df.predator .> 0)
    @test maximum(abs.(conserved .- first(conserved))) < 1e-6
    @test first_local_peak(df.prey, df.t).index > 0
    @test first_local_peak(df.predator, df.t).index > 0
end

```

Листинг 10 — Полный файл `project/test/runtests.jl`.

```
rhktrlwmd@Mac % make test
julia --project=. test/runtests.jl
Test Summary: | Pass Total Time
SIR model      |    5      5 1.5s
Test Summary: | Pass Total Time
Lotka-Volterra model |    5      5 0.4s
rhktrlwmd@Mac %
```

Рисунок 19: Десять успешно пройденных проверок

5. Выводы

В работе реализованы обе предусмотренные методичкой базовые модели и два отдельных параметрических сценария. Для SIR численно подтверждены сохранение населения, условие роста через R_0 и прохождение пика при снижении R_e . Для модели Лотки–Вольтерры получены циклы со сдвигом фаз, замкнутый фазовый портрет, спектральная оценка периода и сохранение первого интеграла.

Сформированы четыре литературных исходника, четыре чистых Julia-скрипта, четыре QMD, четыре HTML-документа и четыре выполненных IPYNB. Базовые и параметрические вычисления сохранили CSV, JLD2 и 24 графических результата. Все десять автоматических проверок завершены успешно.

Список литературы

1. Королькова А. В., Кулябов Д. С. Имитационное моделирование. Практикум. Москва: Российский университет дружбы народов имени Патриса Лумумбы, 2026.

2. Kermack W. O., McKendrick A. G. [A Contribution to the Mathematical Theory of Epidemics](#) // Proceedings of the Royal Society A. 1927. T. 115, № 772. C. 700–721.
3. Hethcote H. W. [The Mathematics of Infectious Diseases](#) // SIAM Review. 2000. T. 42, № 4. C. 599–653.
4. Lotka A. J. [Analytical Note on Certain Rhythmic Relations in Organic Systems](#) // Proceedings of the National Academy of Sciences. 1920. T. 6, № 7. C. 410–415.
5. Volterra V. [Fluctuations in the Abundance of a Species Considered Mathematically](#) // Nature. 1926. T. 118. C. 558–560.
6. Rackauckas C., Nie Q. [DifferentialEquations.jl – A Performant and Feature-Rich Ecosystem for Solving Differential Equations in Julia](#) // Journal of Open Research Software. 2017. T. 5, № 1. C. 15.
7. Datseris G. и др. [DrWatson: the perfect sidekick for your scientific inquiries](#) // Journal of Open Source Software. 2020. T. 5, № 54. C. 2673.
8. Literate.jl contributors. [Literate.jl Documentation](#). 2026.